

C语言与程序设计

The C Programming Language

第6章 编译预处理

王多强

1097412466

dqwang@hust.edu.cn

华中科技大学计算机学院

编译预处理：是在对程序员编写的源程序正式编译之前所作的预处理，由**编译预处理程序**完成。预处理后，“程序员编写的源程序”变成“内部表示的一种源程序”，然后交给编译器进行正式编译。

- 编译预处理是对C程序源码中的**预处理指令**进行处理。
- 编译预处理通常是一个简单的**文本代换**的过程，即由“程序员编写的源程序”变成“内部表示的一种源程序”，都是文本程序，而不是二进制程序。

编译预处理指令：C程序中以“#”号开头的行称为编译预处理指令。

本章介绍三个常用的编译预处理指令：

#include、#define、条件编译

6.1 文件包含指令 #include

文件包含预处理指令有2种一般形式：

(简称**文件包含指令**)

#include <**文件名**>

#include "**文件名**"

这里，**include**是关键字，**文件名**是外存上存在的文件的名称。

功能：用指定文件的内容替换该预处理指令所在的行，即在该行的位置直接粘贴指定文件的内容，使之成为源文件的一部分(**复制-粘贴**，原来的行被直接替换掉)。

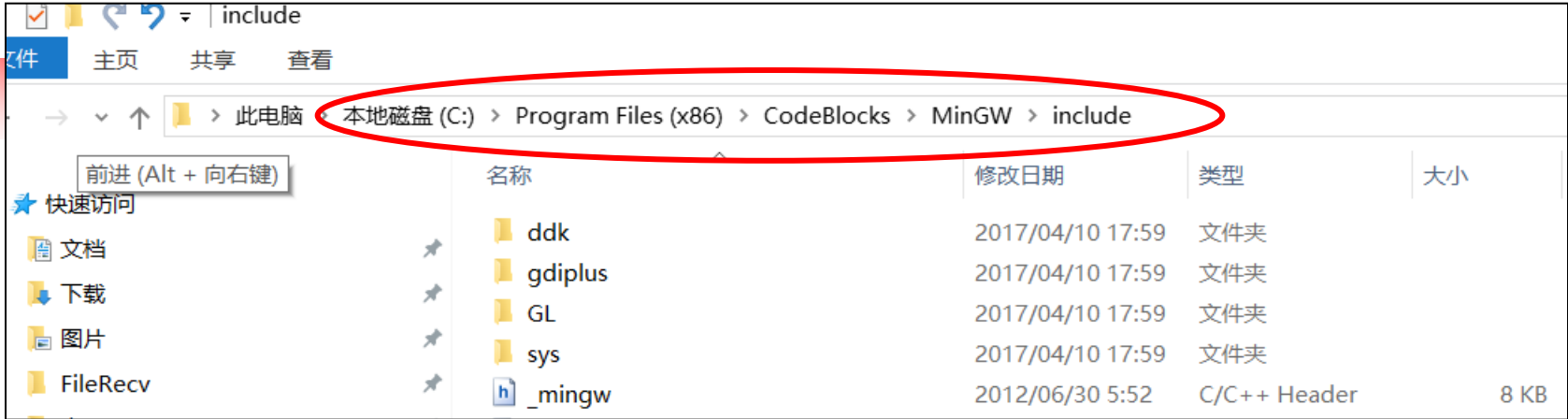
```
#include <文件名>
```

```
#include "文件名"
```

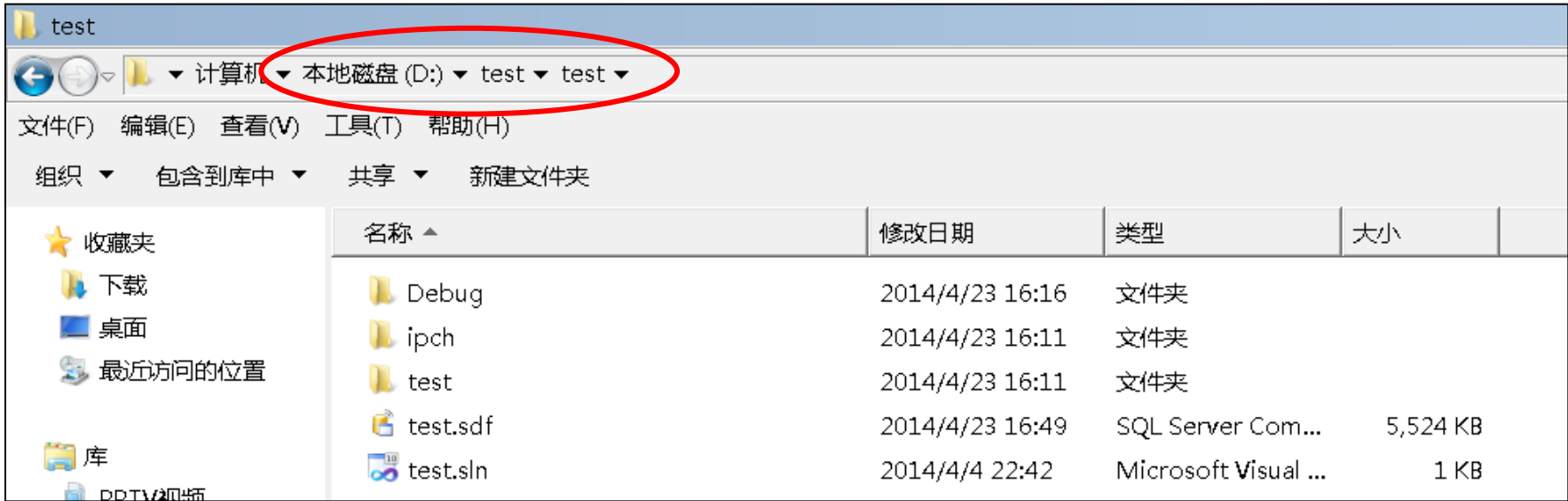
■ 两种形式的区别：

- **#include <文件名>**：在指定的**标准目录**（一般为编译器的安装目录或IDE里设置的路径）下寻找被包含文件，**一般用于标准头文件**，如stdio.h。如果找到则嵌入进来，如果没找到，则报错。
- **#include "文件名"**：首先在**用户工作目录**中寻找被包含的文件，若找不到，再在标准目录下寻找。**一般用于用户自定义的头文件**。如果找到则嵌入进来，如果两个目录下都没找到，则报错。

例: CodeBlocks\MinGW在C盘上的安装目录: C:\Program Files(86)\CodeBlocks\MinGW
其include标准子目录: C:\Program Files(86)\CodeBlocks\MinGW\include



用户工作目录: 就是你的源程序所在的目录





尖括号或者双引号里面的**头文件可以包含指定的路径，此时就直接在指定的路径下找文件**，找到则包含进来，找不到则报错。

如：`#include <d:\test\myfile.h>` 或

`#include "d:\test\myfile.h"`

注：此处目录分割符用**单斜杠** `'\'` 。

被包含文件的内容理论上没有限制，可以是C的源代码，也可以是其它数据文件。但一般是指C语言的**头文件**，即**.h**文件。

头文件：C语言中将扩展名为“**.h**”的文件统称为**头文件**。

- ◆ 因这一类文件通常在 **.c** 源程序中以文件包含命令的方式置于**文件的首部**，所以被称为**头文件**（即**head**文件）。
- ◆ C编译系统约定头文件的扩展名为“**.h**”，但这不是必需的，可以为其它任意名称。

头文件中一般包含：

外部变量声明、外部函数声明、常量声明、用户自定义的数据类型、宏的定义等声明性内容。

□ 头文件的作用——主要用于多文件结构：

- 一个大的程序往往分成几个.c文件编写。
- 每个.c文件中都可能定义有一些全局变量、外部函数为其它.c文件中的函数使用, 还有一些公用的符号常量、类型声明等。
- **C语言要求**：当一个.c文件中的函数引用了另一个.c文件中定义的外部函数、全局变量等时，必须在引用前对这些函数、变量进行外部声明（extern声明）。

方法一：在**每个.c文件中**都写一遍对这些外部符号的声明和共享符号常量的定义，但这样**很麻烦**！

- 有没有简洁的方法？—— **头文件**！

◆ 生成 .h 文件:

将一个 .c 文件中会被公用到**符号常量(宏)定义**、**数据类型定义**、**函数声明**、**外部变量声明**及其它**预处理指令**等单独写成一个 .h 文件。然后在需要使用这些符号的 .c 文件中用 **#include** 命令包含进来, 就可以达到 “**一次书写.h文件、到处使用**” 的目的了。

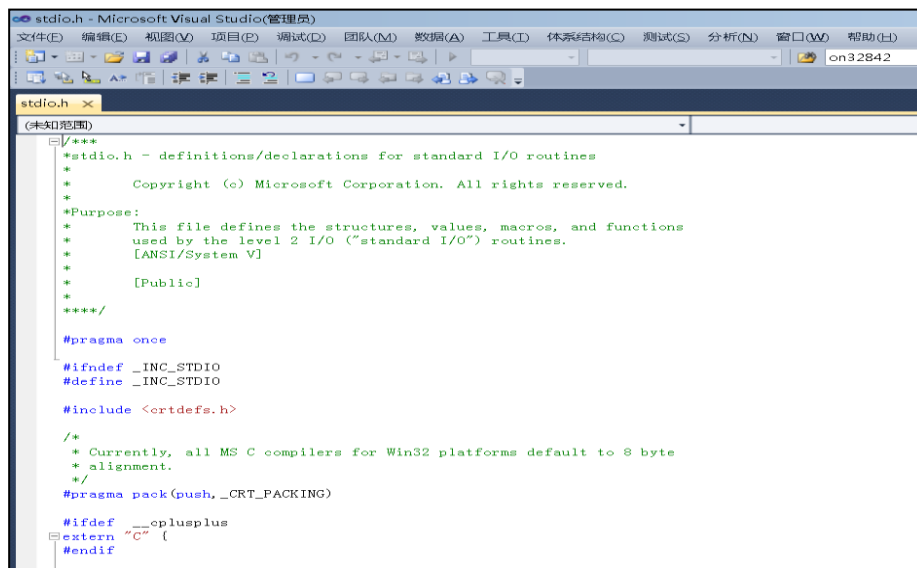
—— 在每个 .c 文件中, 只需写一句**文件包含语句**即可把所有的声明包含进来, 而不用重复写 .h 文件中繁琐的内容, 从而节省了时间, 并且可以减少出错的可能性。

- 通常, 每个 .c 文件都有一个对应的同名 .h 文件。

如, 若有 **myfile.c**, 则通常会有一个 **myfile.h**

stdio.h包含什么内容?

stdio.h包含了C语言提供的**标准输入输出**功能中所定义的常量、函数、宏等的声明，如**printf**、**scanf**函数的声明。

A screenshot of the Microsoft Visual Studio IDE showing the stdio.h header file. The window title is 'stdio.h - Microsoft Visual Studio(管理员)'. The menu bar includes '文件(F)', '编辑(E)', '视图(V)', '项目(E)', '调试(D)', '团队(M)', '数据(A)', '工具(I)', '体系结构(C)', '测试(S)', '分析(N)', '窗口(W)', and '帮助(H)'. The toolbar shows various icons for file operations and development. The code editor displays the following content:

```
stdio.h
(未知范围)

/****
 * stdio.h - definitions/declarations for standard I/O routines
 *
 * Copyright (c) Microsoft Corporation. All rights reserved.
 *
 * Purpose:
 * This file defines the structures, values, macros, and functions
 * used by the level 2 I/O ("standard I/O") routines.
 * [ANSI/System V]
 *
 * [Public]
 *
 *****/

#pragma once

#ifndef _INC_STDIO
#define _INC_STDIO

#include <crtdefs.h>

/*
 * Currently, all MS C compilers for Win32 platforms default to 8 byte
 * alignment.
 */
#pragma pack(push, _CRT_PACKING)

#ifdef __cplusplus
extern "C" {
#endif
```

其它系统头文件，如**stdlib.h**、**math.h**，也都包含了相应库函数的声明等内容。（**自行查看**）

◆ 如果不“包含进来”，编译时系统将“不认得”这些函数。

例：第五章的例子

```
int a=3;
extern int b;
extern void func1(int x, int y);
void otherfunc1( )
{
    .....
    b=2*a;
    .....
}
int otherfunc2(int x)
{
    int y;
    .....
    func1(x, y);
    .....
}
```

file1.c

```
int b=10;
extern int a;
extern void otherfunc1( );
extern int otherfunc2(int x);
int main()
{
    .....
    otherfunc2();
    .....
}
void func1(int x, int y)
{
    .....
    otherfunc1();
    .....
}
```

file2.c

改写:

对每个.c 文件生成一个**同名的.h 文件**，其内容是每个对应.c 文件中需要对外做的外部变量声明和外部函数声明。

file1.h

```
extern int a;  
  
extern int otherfunc1( );  
  
extern int otherfunc2(int x) ;
```

file2.h

```
extern int b;  
  
extern int func1(int x, int y);
```

然后在每个.c文件中把要用到的头文件包含进来，一般也把自己的头文件包含进来（**声明的内容可以重复**，不影响编译）。

```
#include <file1.h>
```

file1.c

```
#include <file2.h>
```

```
int a=3;
```

```
void otherfunc1()
```

```
{ .....
```

```
    b=2*a;
```

```
    .....
```

```
}
```

```
int otherfunc2(int x)
```

```
{
```

```
    int y;
```

```
    .....
```

```
    func1(x, y);
```

```
    .....
```

```
}
```

```
#include <file2.h>
```

file2.c

```
#include <file1.h>
```

```
int b=10;
```

```
int main()
```

```
{
```

```
    .....
```

```
    otherfunc2();
```

```
    .....
```

```
}
```

```
void func1(int x, int y)
```

```
{
```

```
    .....
```

```
    otherfunc1();
```

```
    .....
```

```
}
```

6.2 宏定义 #define

在C语言源程序里用下述方式定义标识符称为**宏定义**：

#define 宏名 字符序列

如：**#define PI 3.1415926**

其中：

符号常量是宏的一种

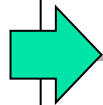
- **define**：关键字
- **宏名**：宏的名字，必须为合法的标识符。
- **字符序列**：宏的内容，为任意字符的序列。宏名和字符序列简称为**宏串**之间至少用一个分隔符（空格）隔开。

宏的作用：在编译预处理时，对程序中出现的所有宏名用宏定义中的字符序列进行**代换**（称为**宏代换**或**宏展开**）。

例： 预处理前

```
#define M (y*y+3*y )
```

```
int main(void)
{
    int s,y;
    printf("Input a number: ");
    scanf("%d",&y);
    s=3*M+4*M+y*M;
    printf("s=%d\n",s);
    return 0;
}
```



预处理后

```
int main(void)
{
    int s,y;
    printf("Input a number: ");
    scanf("%d",&y);
    s=3* (y*y+3*y ) +4* (y*y+3*y )
      +y* (y*y+3*y );
    printf("s=%d\n",s);
    return 0;
}
```

注：宏展开只是一个简单的**文本替换**过程：**用宏串替换宏名**。

6.2.1 无参宏定义

- **无参宏**：不带参数的宏，是最基本的宏定义形式。

一般形式：**#define** **标识符** **字符序列**

如：`#define EPS 1.0e-9`

`#define MAX_INT 32767`

- **宏名一般用大写**

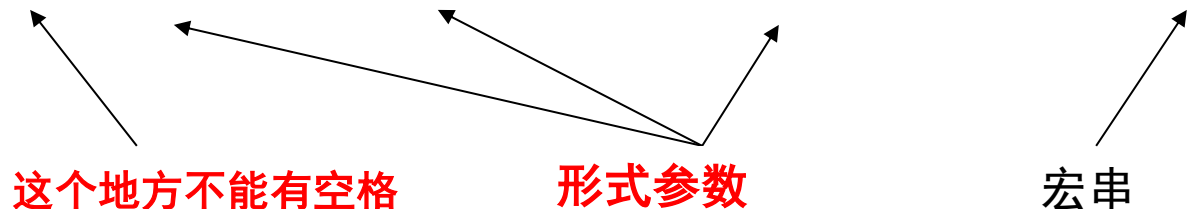
- 如果字符序列太长，一行写不下，可以用续行符(\)分成多行书写。

- **应用**：常用于定义**符号常量**，如圆周率 π 、光速 c 等
- **好处**：使用符号常量可以改善程序的**可读性**和**可移植性**

6.2.2 带参数的宏定义

- 带参数的宏形式上像函数（故也称为**宏函数**），定义方式如下：

#define 标识符(标识符1, 标识符2, ..., 标识符k) 字符序列



例 定义计算 x^2 的宏: **#define square(x) ((x)*(x))**

若有**宏调用**: **square(a+1)**

则**宏展开**为: **((a+1) * (a+1))**

带参数的宏的调用: 形式上如函数调用，但处理方式不同：引用时给出实参，宏展开时**首先用宏串替换宏名，然后用实参去替换形参，最后形成新的源码嵌入源程序中，再进行编译。**

思考：为什么要用这么多的括号？ `#define square(x) ((x)*(x))`

设有宏： `#define SQ(x) x*x`

若有宏调用： `SQ(a+b)`

则宏展开是： `a+b*a+b` /* 与 `(a+b)*(a+b)` 不同 */

再设： `#define SQ(x) (x)*(x)`

若有宏调用： `27/SQ(3)`

则宏展开是： `27/(3)*(3)` /* 值27, 与 `27/32` 不同 */

再次强调：宏展开只是简单的文字代换处理。因此，在定义带参数的宏时，为了保证计算次序的正确性，**表达式中的每个参数都应该用括号括起来，整个表达式也要用括号括起来。**这样宏展开时才能确保逻辑上正确。

其它注意事项:

(1) 宏名和参数表的左括号之间不能有空格

如: `#define SQ (x) ((x)*(x))`

- ◆ 否则, 编译器认为该宏是**无参宏定义**。如上例, SQ代表的宏串将被认为是 `(x) ((x)*(x))`

若此时宏调用: `SQ(3)`

则宏展开为: `(x) ((x)*(x)) (3)` ✗ /*显然错误的*/

(2) 带参数的宏可以嵌套调用: `#define square (x) ((x)*(x))`

若有: `square (square (a))`

依次进行宏展开: `((((a)*(a))) * (((a)*(a))))`

(3) 除非分号是宏串的一部分，否则宏串尾部不要加分号。

例： 预处理前

```
#define M (y*y+3*y);
```

```
int main(void)
```

```
{ int s,y;
```

```
    printf("Input a number: ");
```

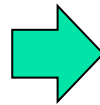
```
    scanf("%d",&y);
```

```
    s=3*M+4*M+y*M;
```

```
    printf("s=%d\n",s);
```

```
    return 0;
```

```
}
```



预处理后

```
int main(void)
```

```
{ int s,y;
```

```
    printf("Input a number: ");
```

```
    scanf("%d",&y);
```

```
    s=3* (y*y+3*y ); +4* (y*y+3*y );
```

```
        +y* (y*y+3*y );
```

```
    printf("s=%d\n",s);
```

```
    return 0;
```

```
}
```



(4) 用**单引号或双引号**括起来的与宏名相同的字符序列不会被替换。它们仅被当作字符或字符串的内容，不被视为宏名。

如： **#define OK 100**

printf("OK");

(5) “宏函数”一般只适用于处理简单的“函数”，复杂函数还是要写成真正的函数为好。

(6) 预处理程序进行**宏展开时不作任何语法、语义的合法性及错误方面的检查**。因此，即使有错误，也只能在宏展开后，在编译源程序的时候由编译器去发现和处理，并且**指出的错误在宏引用处**，而不是宏定义处。因此这些错误对程序调试具有隐蔽性，一般要慎重使用。

另外，带参的宏通常被认为不安全（自学）

- ◆ 宏由预处理器执行，而预处理器并不懂得C，可能会不小心而引入一些错误

如 **#define SQ (x) ((x)*(x))**

无法及时发现错误

- ◆ 有些错误编译器也发现不了

如： **#define SQ(x) ((x)*(x))**

宏调用： **SQ(++a)**

宏展开： **((++a)*(++a))**

这里a被加2次。如是函数调用就不会有此问题（函数调用a只会被加1次）。

不过，尽管带参的宏有不安全性，但还是很有价值

◆ 相比函数：

- **优点**：宏节省了函数调用的开销，程序运行速度更快；
宏的形式参数不分配内存单元，不必作类型说明。
- **缺点**：宏展开后使源程序和执行代码长度有所增长。

◆ 慎用宏：

- **宏比较适合于经常使用的简短表达式，以及小的可重复的代码段；**
- 当任务比较复杂，需要多行代码才能实现时，或者要求程序越小越好时，就应该使用函数。

6.2.4 空宏参数 (自学)

C99允许宏调用中任意或所有参数为空。

例如：

```
#define ADD(x,y) ((x)+(y)) /*定义求和的宏*/
```

```
若有： x = ADD(2,3);
```

```
      y = ADD( ,3);          /* 第1个参数为空*/
```

预处理后变为：

```
x = (2+3); /* x的值为5 */
```

```
y = (+3); /* y的值为3 */
```


6.2.5 可变参数宏定义 (自学)

C99增加了可变参数宏 (variadic macros) , 允许像下面

这样定义可变参数宏:

```
#define debug(format, ...) printf (format, __VA_ARGS__)
```

这里, debug是一个可变参数宏, format是宏的一个参数, **省略号代表一个能够改变的参数表**。在每次被调用时, “...” 被表示成零个或多个参数。内建的预处理器标识符__VA_ARGS__ 用来把 “...” 部分传递给宏。当宏的调用展开时, 实参就取代__VA_ARGS__。

例如, 宏调用

```
debug("x= %d\n, y=%d\n",10,20); /* 输出 x=10, y=20 */
```

会被展开成:

```
printf("x= %d\n, y=%d\n",10,20);
```

【例6.2】 用C99的可变参数宏，打印调试信息。

在编写代码的过程中，为了调试程序，经常会输出一些调试信息到屏幕上。随着项目的调试，输出的信息可能会越来越多，信息的输出一般要调用printf等函数，但是当调试完后，又需要手工将这些地方删除或者注释掉。这样做工作量比较大，很麻烦。如何方便地处理这些用于输出调试信息的语句？

用C99的可变参数宏，可方便地输出调试信息。

```
#include <stdio.h>
```

```
#define DEBUG
```

```
#ifdef DEBUG
```

```
#define msg(fmt, ...) printf(fmt, __VA_ARGS__)
```

```
#else
```

```
#define msg(fmt,...)
```

```
#endif
```

```
int main()
```

```
{
```

```
    printf("hello!\n");
```

```
    msg("%s %d %5d\n", "debug1", 10, 20);
```

```
    msg("%s %s %10.2f %d\n", "debug2", "test", 10.5, 100);
```

```
    return 0;
```

```
}
```

调试阶段定义DEBUG宏，在需要输出调试信息的地方用宏msg，调试成功后，软件正式发行时，只需将第2行的#define指令删除或注释掉即可，非常方便。

6.2.6 通用类型宏（自学）

- ◆ 通用类型宏或者泛型宏（type-generic macros）是一种编译期技术，它允许开发人员根据宏的某个参数的类型来确定生成的内容。早在C99时就有了通用类型宏的概念，只不过当时没有对它进行标准化。
- ◆ C99中引用了头文件<tgmath.h>，给开发人员提供大量初等数学函数接口。如在C99中，程序员可以用不同的类型来调用sin函数，比如：
 - sin(1)，实际上调用sin(1.0);
 - sin(1.0F)，实际上调用sinf(1.0F);
 - sin(1.0L)，实际上调用sinl(1.0L)。

实际上，sin这个接口是一个宏，它会根据传来的实际参数类型展开成特定的函数。sin就是一个通用类型宏。

C11中引入了一个新关键字 **Generic** 来实现通用类型宏，它根据第一个参数的类型和后面的类型-表达式关联来实现编译期的替换。

利用 `_Generic`，C99的 `sin` 可定义如下：

```
#define sin(x) _Generic ( (x), long double: sinl, double: sinf, default: sin ) (x)
```

这里，`_Generic`对第一个参数进行类型判断，然后根据从第二参数开始的类型-表达式关联表来进行编译期替换。如果 `x` 为 `long double` 类型，那么 `_Generic((x), ...)` 的结果为 `sinl`，如果 `x` 为 `double` 类型，那么 `_Generic((x),...)` 的结果为 `sinf`，否则结果为 `sin`。可见，根据 `x` 的类型，宏 `sin(x)` 转换为 `sinl(x)`，`sinf(x)` 或 `sin(x)`。

【例6.3】 用_Generic, 编写求和的通用类型宏sum。

```
int sumi(int *arr, int cnt)      /* 整数求和 */
{
    int sum = 0;
    int i;
    for(i = 0; i < cnt; ++i) sum += arr[i];
    return sum;
}

double sumf(double *arr, int cnt) /* 浮点数求和 */
{
    double sum = 0.0;
    int i;
    for(i = 0; i < cnt; ++i) sum += arr[i]
    return sum;
}

/* 通用类型宏sum, 它会根据传递的实际类型来决定最终调用的函数 */
#define sum(_arr, _cnt) _Generic(_arr[0],int: sumi, default: sumf)(_arr, _cnt)
```

6.2.3 取消宏定义#undef

- ▣ **取消宏**：终止宏名的作用域，形式为：

#undef 标识符

- 其中标识符为#define定义过的宏名
- 如果没有取消宏，则**宏名的作用域从其定义点开始直到宏定义指令所在的文件结束处为止。**
- 宏取消后，不能再被引用，否则出错。

▣ 何时使用#undef指令?

(1) 防止宏名的冲突

```
#include "everything.h"
```

```
#undef SIZE    /*若everything.h中定义了SIZE, 就取消它; 否则该指令不起作用*/
```

```
#define SIZE 100    /*重新定义*/
```

■ 保证调用的是一个实际函数而不是宏

```
#undef getchar
```

```
int getchar(void) {...}
```


6.3 条件编译

对源程序的某些部分进行有选择的编译，称为**条件编译**。

条件编译有三种指令形式：

形式1：#if指令

```
#if 常量表达式
    程序段1
[#elif 常量表达式
    程序段... ]
...
[#else
    程序段2 ]
#endif
```

形式2：#ifdef指令

```
#ifdef 标识符
    程序段1
[#elif 常量表达式
    程序段... ]
...
[#else
    程序段2 ]
#endif
```

形式3：#ifndef指令

```
#ifndef 标识符
    程序段1
[#elif 常量表达式
    程序段... ]
...
[#else
    程序段2 ]
#endif
```

这里条件选择的控制逻辑与if语句类似。

形式1: #if指令

#if 常量表达式

程序段1

[**#elif** 常量表达式

程序段...]

...

[**#else**

程序段2]

#endif

含义：若**#if**后面的常量表达式为非0，则编译**程序段1**，否则（若有**#elif**，则形成嵌套的条件选择，依次处理；若没有**#elif**，则）处理**#else**，编译**程序段2**。

- ◆ **常量表达式**必须是**整型**，非零代表TRUE，0代表FALSE。
- ◆ **#if**和**#endif**配对，**#endif**不能少。
- ◆ **#if**、**#elif**、**#else**、**#endif**是条件编译指令，书写的时候**#**不能少。
- ◆ **#elif**、**#else**部分是可选的。

形式2: #ifdef 指令

#ifdef 标识符

程序段1

[#elif 常量表达式

程序段...]

...

[#else

程序段2]

#endif

- ◆ **标识符**是通过**#define**定义的**宏名**。

- ◆ **含义:**

若**#ifdef**后面的标识符在**之前被#define命令定义过**，则编译**程序段1**，否则（若有**#elif**，则形成嵌套的条件选择，依次处理；若没有**#elif**，则）处理**#else**，编译**程序段2**。

- **标识符**可以存在也可以不存在，但若存在，就必须是通过**#define**定义过的**宏名**。
- **其它要求和#if指令相同**。

形式3: #ifndef指令

#ifndef 标识符

程序段1

[**#elif** 常量表达式

程序段...]

...

[**#else**

程序段2]

#endif

- ◆ **标识符**是通过**#define**定义的**宏名**。

- ◆ **含义:**

若 **#ifndef** 后面的标识符在**之前** 没有被**#define**命令定义过, 则编译**程序段1**, 否则 (若有**#elif**, 则形成嵌套的条件选择, 依次处理; 若没有**#elif**, 则) 处理**#else**, 编译**程序段2**。

- **标识符**可以存在也可以不存在, 但若存在, 就必须是通过**#define**定义过的**宏名**。

- **其它要求和#ifdef指令相同。**

▣ 条件编译的预处理方法

在处理条件编译指令时，**根据条件，保留需要编译的代码部分，而不需要编译的代码直接从源程序中删除。**

删减后的源程序作为条件编译预处理的结果交给编译器正式编译。这样条件编译指定的“**不编译**”部分就被舍弃了。

例6.4 利用条件编译计算圆或正方形的面积

预处理前

```
#define R
```

```
int main(void)
{ float c, r, s;
  printf ( "input a number: " );
  scanf( "%f" , &c);
```

```
#ifdef R
```

```
    r=3.14159*c*c;
```

```
    printf( "%f\n" , r);
```

```
#else
```

```
    s=c*c;
```

```
    printf("%f\n", s);
```

```
#endif
```

```
return 0;
```

```
}
```

这一部分在预处理时被舍去

预处理后

```
int main(void)
```

```
{
```

```
    float c, r, s;
```

```
    printf ( "input a number: " );
```

```
    scanf( "%f" , &c);
```

```
    r=3.14159*c*c;
```

```
    printf( "%f\n" , r);
```

```
return 0;
```

```
}
```

只留下了要编译的源代码，不要编译的源代码直接舍弃。所以可执行代码里也不会有被舍弃部分的二进制代码。

□ 条件编译的作用

(1) 可根据不同的平台，如Windows、Linux，编译不同的部分，从而产生不同的目标代码。如：

```
#define LINUX
```

```
#ifdef LINUX
```

Linux程序代码

```
#else
```

Windows程序代码

```
#endif
```

(2) 调试和正式发布可分别使用不同的代码，如调试信息的处理。如：

```
#define DEBUG
```

```
#ifndef DEBUG
```

正式发布的代码

```
#else
```

调试用的代码

```
#endif
```

defined运算符

- ▣ **defined**: **预处理运算符**, 用于判断标识符是否被#define过。

表达式形式: **defined(标识符)** 或 **defined 标识符**

表达式结果: 若标识符被定义, 值为**1**(TRUE), 否则为**0**(FALSE)

- ▣ **作用**: 类似 **#if defined(R)** 形式的应用, 可用于写比较复杂的条件编译指令:

如, 有两个宏**MACRO_1**和**MACRO_2**, 只有两个宏都定义过才会编译程序段A, 可通过如下方式实现:

```
#if defined(MACRO_1) && defined(MACRO_2)
```

程序段A

```
#endif
```


例6.5 采用条件编译，避免多次包含同一个头文件。

为了避免一个头文件被多次包含，可以在头文件的最前面两行和最后一行加上预编译指令，让头文件在被多个源文件引用时不会多次编译。

```
#ifndef _NAME_H
#define _NAME_H          /* 定义头文件的标识符 */
.....                  /* 头文件的内容 */
#endif
```

其中，NAME是头文件的名字。比如头文件为myFile.h，则其标识符可为_MYFILE_H。

■ 说明：

在创建一个头文件时，用#define指令为它定义一个唯一的标识符。通过#ifndef指令检查这个标识符是否已被定义。如果已被定义，则说明该头文件已经被包含了，就不要再次包含该头文件，#ifndef帮助编译器跳过直到#endif的所有文本；反之，则定义这个标识符，以避免以后再次包含该头文件。

例6.6 条件编译允许有选择地编译程序的某些部分，可以将程序的特殊性能纳入不同版本。

例如对于不同语言版本中的某个应用程序，需要改变货币的显示，可以使用以下条件编译，使用预定义常数 `ACTIVE_COUNTRY` 的值来决定货币符号。

```
#define US          0

#define ENGLAND      1

#define FRANCE       2

#define ACTIVE_COUNTRY US

#if ACTIVE_COUNTRY == US

    char currency[ ]= "dollar";    /* 美元 */

#elif ACTIVE_COUNTRY==ENGLAND

    char currency[ ]= "pound";     /* 英镑 */

#else

    char currency[ ]= "franc";     /* 法郎 */

#endif
```

▣ 调试程序时临时忽略一些代码

在程序开发过程中，程序员经常需要临时忽略或封闭一些代码，从而防止编译器编译这些代码。要做到这一点，可以把代码放在注释中。然而，如果代码中也含有注释，这个方法就会导致语法错误（注释不能嵌套）。

◆ 使用条件编译能解决这个问题：

#if 0

不编译的代码

#endif

要让编译器编译这段代码，把原来的0改为1就可以了。

调试程序时跟踪程序的执行

在源程序的调试中，常常需要跟踪程序的执行情况，为此，在程序上加一些输出信息的语句，通过这些输出信息来跟踪判断程序是否有错误。在调试结束后，需要把为了调试而增加的那些输出信息的语句删除掉，然而手工删除既不方便，也易出错。**使用条件编译方便得多，把这些为调试而增加的语句放在条件编译指令之间，在调试时编译这些语句。**

如：**#ifdef DEBUG**

```
printf("Variable x=%d\n",x);
```

#endif

在调试程序时，在前面加#define DEBUG，就编译printf语句，输出供判断参考的x值。完成调试后，从程序中去掉#define指令，编译就忽略为调试而插入的printf语句，相当于它被“自动”删除了。

6.4 assert断言和静态断言（自学）

断言是**异常处理**的一种高级形式，用于在代码中捕捉一些假设，当假设不成立时中断当前操作；

- ◆ 使用断言可以创建更稳定、品质更好且不易于出错的代码。
- ◆ **assert断言**是**动态断言**，只能在程序运行出现错误时做出判断。C11增加了**静态断言**，它可以在编译时就对程序的错误做出判断。

6.4.1 assert断言

- ◆ **assert**的声明在头文件**assert.h**中
- ◆ **形式**: **assert(condition)**
- ◆ **作用**: 用来测试表达式的值是否符合要求
 - ◆ 如果**condition值非0**, 程序继续执行下一个语句。
 - ◆ 如果**condition值0**, 就输出错误信息, 并通过调用实用库中的函数**abort**终止程序的执行。

例：用assert宏判断数据是否合法

assert (x<=10);

- ◆ 如果x大于10，就会输出如下包含行号和文件名的错误信息并中断执行：

Assertion failed:x<= 0,file test.c,line 12

对于大多数编译器来说，在头文件assert.h的**assert**宏定义中，如果定义了符号常量NDEBUG，其后的assert将被忽略。因此，如果不再需要assert，那么可把代码行

#define NDEBUG

插入到程序中，而无需手工删除assert。

6.4.2 静态断言

assert宏只能在程序运行出现错误时进行退出操作并产生调试信息，而**静态断言**（Static assertions）可用于在编译时进行检查，不会产生任何运行时的额外开销（包括时间和空间）。

在C11标准中，从语言层面加入了对静态断言的支持，引入了新的关键字 `_Static_assert` 来表示静态断言，断言失败会产生有意义的且充分的诊断信息。

关键字 `_Static_assert`

`_Static_assert (constant-expression, string-literal);`

- ◆ 第一个参数constant-expression必须是一个编译时可知的整型常量表达式，如果用一个变量作为第一个参数会遇到编译错误；第二个参数string-literal是在断言失败时输出的提示信息（即字符串）。
- ◆ 当constant-expression的布尔值为true时，该静态断言声明不会产生任何影响；否则，编译器将给出错误诊断信息string-literal。

例如：

```
_Static_assert(sizeof(int) == 8, "A 64-bit machine needed!");
```

在32位机上编译这条语句时，就会输出如下诊断信息：

```
static assertion failed: "A 64-bit machine needed!"
```

注：在头文件assert.h中，定义static_assert宏为关键字_Static_assert的同义词。

6.5 _func_ 预定义标识符

C99标准中引入了预定义标识符（**predefined identifier**）的概念，并定义了一个预定义标识符 **_func_**。它的性质和关键字相似，但它本身并不是关键字。

- ▣ **_func_** 定义为**字符数组**，用于指出 **_func_** 所在的**函数名**，类似于字符串赋值。例如：

```
#include <stdio.h>
void myfunc(void)
{
    printf("%s\n", __func__);
    /* ... */
}
```

每次调用函数myfunc时，
都输出：**myfunc**。

6.6 _Pragma预处理操作符

C99标准中增加了 **_Pragma** 操作符，以便更灵活地利用

pragma工具，其使用形式如下：

_Pragma(字符串常量)

含义： 它把字符串常量的内容（把字符串常量内部的\"替换为\"，把\\替换为\\之后）看成是**#pragma**指令中所出现的预处理器标记。

例如：

_Pragma("OPTIMIZE OFF") /* 注意：后面是没有分号的 */

与 **#pragma OPTIMIZE OFF** /* 指示编译器停止优化代码 */

是一样的。

- ▣ 与#pragma相比，_Pragma的优势在于可以用于宏定义。

本章小结

- ◆ 预处理指令以 “#” 开头。常用的有三个：`#include`、`#define` 和条件编译。
 - **#include**把指定的文件包含到程序中。
 - **#define**用来定义宏，有带参数和不带参数的宏，使用宏可以减少程序的执行时间，增加程序的可读性。
 - 条件编译使程序员能够控制预处理指令的执行和程序代码的编译。
- ◆ 编译前的预处理功能是C语言区别于其他高级语言的一个重要特征之一。